

GENERATIVE GRAMMARS

a.y. 2026-2027

1 / 40

Generative grammars, informally

- Fix a **vocabulary**:
 - A set of symbols: **terminals** and **non-terminals**
 - Terminals play the tokens of the output stream of lexical analysis
- E.g., take the vocabulary $\{S, a, b\}$ where a and b are terminals, and S is non-terminal

2 / 40

Generative grammars, informally

- One non-terminal symbol of the vocabulary is chosen as **start symbol**
- E.g., if the vocabulary is $\{S, a, b\}$, and S is the single non-terminal in the set, then S is surely the start symbol

3 / 40

Generative grammars, informally

- Fix a set of **productions**:
 - Rules for rewriting strings into strings
 - With the constraint that the string to be replaced must contain at least a non-terminal
- E.g., $\{S \rightarrow aSb, S \rightarrow ab\}$, where the arrow $\rightarrow$ means that the string at its left can be rewritten into the string at its right

4 / 40

Generative grammars, informally

- Vocabulary, start symbol, and productions are the ingredients of a **generative grammar**
- A **language**, that is a set of words over the alphabet of terminals, can be generated from the start symbol using the following procedure:
 - Apply the rewriting rules in any possible way, and as many times as possible
 - Each rewriting is called a **derivation step**

5 / 40

Generative grammars, informally

EXAMPLE: $\{S \rightarrow aSb, S \rightarrow ab\}$

- Take the grammar with vocabulary $\{S, a, b\}$, start symbol S , and productions in the set $\{S \rightarrow aSb, S \rightarrow ab\}$
- Then a one-step derivation is:
 - $S \Rightarrow ab$
 - Where $\Rightarrow$ is the notation for the **derivation relation**
- The derived word ab is made up of terminals only
- Hence ab **belongs** to the language generated by the given grammar

6 / 40

Generative grammars, informally

EXAMPLE: $\{S \rightarrow aSb, S \rightarrow ab\}$

- An example of two-steps derivation is:
 - $S \Rightarrow aSb \Rightarrow aabb$
- The derived word *aabb* is made up of terminals only
- Hence *aabb* **belongs** to the language generated by the given grammar

7 / 40

Generative grammars, informally

EXAMPLE: $\{S \rightarrow aSb, S \rightarrow ab\}$

- Another example of two-steps derivation is:
 - $S \Rightarrow aSb \Rightarrow aaSbb$
- The derived word *aaSbb* contains the occurrence of a non-terminal
- Hence *aaSbb* **does not belong** to the language generated by the given grammar

8 / 40

Generative grammars, informally

EXAMPLE: $\{S \rightarrow aSb, S \rightarrow ab\}$

- What is the language generated by the given grammar?
- $\{a^n b^n \mid n > 0\}$

9 / 40

Notation and conventions

- Non-terminals are denoted by capital letters
- Productions are written as a list, one production per line
- The start symbol of the grammar is the symbol at the leftmost position of the production listed as first
- The special character ϵ (“epsilon”) is used to denote the empty word
- The length of ϵ is 0
 - $\epsilon \equiv \epsilon\epsilon$
 - $\epsilon \equiv b^0$ for every terminal b

10 / 40

Example 1

$$\begin{aligned} S &\rightarrow aAb \\ aA &\rightarrow aaAb \\ A &\rightarrow \epsilon \end{aligned}$$

- Generated language: $\{a^n b^n \mid n > 0\}$
- LESSON LEARNT: Different grammars can generate the same language

11 / 40

Example 2

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aA \\ A &\rightarrow a \\ B &\rightarrow Bb \\ B &\rightarrow b \end{aligned}$$

- Generated language: $\{a^n b^m \mid n, m > 0\}$

12 / 40

Example 3

$$\begin{aligned} S &\rightarrow aSBc \\ S &\rightarrow abc \\ cB &\rightarrow Bc \\ bB &\rightarrow bb \end{aligned}$$

- Generated language: $\{a^n b^n c^n \mid n > 0\}$

13 / 40

Example 4

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow a \end{aligned}$$

- Generated language: $\emptyset$

14 / 40

Example 5

$$S \rightarrow \epsilon$$

- Generated language: $\{\epsilon\}$
- $\{\epsilon\} \neq \emptyset$

15 / 40

Example 6

$$\begin{aligned} S &\rightarrow aSb \\ S &\rightarrow \epsilon \end{aligned}$$

- Generated language: $\{a^n b^n \mid n > 0\} \cup \{\epsilon\} = \{a^n b^n \mid n \geq 0\}$

16 / 40

Example 7

$$\begin{aligned}
 S &\rightarrow CD \\
 C &\rightarrow aCA \\
 C &\rightarrow bCB \\
 AD &\rightarrow aD \\
 BD &\rightarrow bD \\
 Aa &\rightarrow aA \\
 Ab &\rightarrow bA \\
 Ba &\rightarrow aB \\
 Bb &\rightarrow bB \\
 C &\rightarrow \epsilon \\
 D &\rightarrow \epsilon
 \end{aligned}$$

- Generated language?

17 / 40

Example 7

Derivation 1

$$\begin{aligned}
 S &\rightarrow CD \\
 C &\rightarrow aCA \\
 C &\rightarrow bCB \\
 AD &\rightarrow aD \\
 BD &\rightarrow bD \\
 Aa &\rightarrow aA \\
 Ab &\rightarrow bA \\
 Ba &\rightarrow aB \\
 Bb &\rightarrow bB \\
 C &\rightarrow \epsilon \\
 D &\rightarrow \epsilon
 \end{aligned}$$

$$\begin{aligned}
 \underline{S} &\Rightarrow CD \quad (p1) \\
 \underline{C}D &\Rightarrow D \quad (p10) \\
 \underline{D} &\Rightarrow \epsilon \quad (p11)
 \end{aligned}$$

18 / 40

Example 7

Derivation 2

$$\begin{aligned}
 S &\rightarrow CD \\
 C &\rightarrow aCA \\
 C &\rightarrow bCB \\
 AD &\rightarrow aD \\
 BD &\rightarrow bD \\
 Aa &\rightarrow aA \\
 Ab &\rightarrow bA \\
 Ba &\rightarrow aB \\
 Bb &\rightarrow bB \\
 C &\rightarrow \epsilon \\
 D &\rightarrow \epsilon
 \end{aligned}$$

$$\begin{aligned}
 \underline{S} &\Rightarrow CD & (p1) \\
 \underline{C}D &\Rightarrow aCAD & (p2) \\
 a\underline{C}AD &\Rightarrow aCaD & (p4) \\
 a\underline{C}aD &\Rightarrow aaD & (p10) \\
 aa\underline{D} &\Rightarrow aa & (p11)
 \end{aligned}$$

Example 7

Derivation 3

$$\begin{aligned}
 S &\rightarrow CD \\
 C &\rightarrow aCA \\
 C &\rightarrow bCB \\
 AD &\rightarrow aD \\
 BD &\rightarrow bD \\
 Aa &\rightarrow aA \\
 Ab &\rightarrow bA \\
 Ba &\rightarrow aB \\
 Bb &\rightarrow bB \\
 C &\rightarrow \epsilon \\
 D &\rightarrow \epsilon
 \end{aligned}$$

$$\begin{aligned}
 \underline{S} &\Rightarrow CD & (p1) \\
 \underline{C}D &\Rightarrow aCAD & (p2) \\
 a\underline{C}AD &\Rightarrow abCBAD & (p3) \\
 ab\underline{C}BAD &\Rightarrow abCBA & (p11) \\
 ab\underline{C}BA &\Rightarrow abBA & (p10)
 \end{aligned}$$

Example 7

Derivation 4

$S \rightarrow CD$	$\underline{S} \Rightarrow CD \quad (p1)$
$C \rightarrow aCA$	$\underline{C}D \Rightarrow aCAD \quad (p2)$
$C \rightarrow bCB$	$a\underline{C}AD \Rightarrow abCBAD \quad (p3)$
$AD \rightarrow aD$	$abC\underline{B}AD \Rightarrow abCBaD \quad (p4)$
$BD \rightarrow bD$	$abC\underline{B}aD \Rightarrow abCaBD \quad (p8)$
$Aa \rightarrow aA$	$abCa\underline{B}D \Rightarrow abCabD \quad (p5)$
$Ab \rightarrow bA$	$abC\underline{a}bD \Rightarrow ababD \quad (p10)$
$Ba \rightarrow aB$	$abab\underline{D} \Rightarrow abab \quad (p11)$
$Bb \rightarrow bB$	
$C \rightarrow \epsilon$	
$D \rightarrow \epsilon$	

21 / 40

Generative grammars, formally

- A grammar is a tuple
 $(V, T, S, \mathcal{P})$
- Where:
 - V is a vocabulary of terminals and non-terminals
 - T is the set of terminals
 - S is the start symbol, taken in $V \setminus T$
 - $\mathcal{P}$ is a set of productions

22 / 40

Notational conventions

- Uppercase letters, early in the alphabet, stand for non-terminals.
E.g., $A, B, \dots \in (V \setminus T)$
- Uppercase letters, late in the alphabet, stand for elements of the vocabulary.
E.g., $X, Y, \dots \in V$
- Lowercase letters, early in the alphabet, stand for terminals.
E.g., $a, b, \dots \in T$
- Lowercase letters, early in the Greek alphabet, stand for strings of elements of the vocabulary.
E.g., $\alpha, \beta, \dots \in V^*$
(Given any set H , the notation H^* stands for **zero or more** concatenations of elements of H . Hence α and β can also be ϵ .)
- Lowercase letters, late in the alphabet, stand for strings of terminals.
E.g., $x, y, \dots \in T^*$

23 / 40

Productions

- The general form of a production is
$$\delta \rightarrow \beta$$
- Where:
 - $\delta \in V^+$
(Given any set H , the notation H^+ stands for **one or more** concatenations of elements of H . Hence δ cannot be ϵ .)
 - δ contains at least a non-terminal
 - δ is called the **driver** of the production
 - β is called the **body** of the production

24 / 40

Generated languages

- Given the grammar

$$\mathcal{G} = (V, T, S, \mathcal{P})$$

- The language generated by $\mathcal{G}$ is denoted $\mathcal{L}(\mathcal{G})$ and is defined as follows:

$$\mathcal{L}(\mathcal{G}) = \{w \mid w \in T^* \text{ and } S \Rightarrow^* w\}$$

25 / 40

Context-free grammars

- Depending on the shape of their productions, formal grammars are classified in a hierarchy
- Our focus is on **context-free grammars** (also simply called **free grammars**)
- A grammar is context-free if each of its production has the shape

$$A \rightarrow \beta$$

- Namely if the driver of each of its productions is simply a non-terminal (A instead of the generic string δ)

26 / 40

Context-free languages

- $\mathcal{L}$ is a **context-free language**
- iff
- there exists a context-free grammar $\mathcal{G}$ such that $\mathcal{L} = \mathcal{L}(\mathcal{G})$.

27 / 40

Context-free languages

- Context-free languages have a number of handy properties
- And, in fact, they are our focus

28 / 40

Canonical derivations

- In a context-free grammar, every production rewrites a single non-terminal (the driver)
- A **rightmost** (resp. **leftmost**) derivation step is a step in which the rewriting is applied to the rightmost (resp. leftmost) non-terminal
- The derivation of a word in the language generated by the grammar is called **canonical** if
 - Either every step in the derivation is rightmost
 - Or every step in the derivation is leftmost

29 / 40

Derivation trees

- The start symbol of the grammar is the root of the tree
- If the production $A \rightarrow X_1X_2 \dots X_n$ is used in the derivation step to rewrite A
- Then generate the children nodes $X_1X_2 \dots X_n$ for node A in the tree
- Terminals are the leaves of the tree (and so is ϵ)
- The derived word is at the **frontier** of the tree

30 / 40

Example

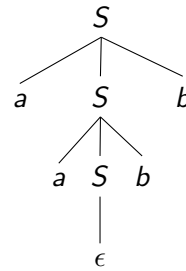
Grammar:

$$S \rightarrow aSb$$
$$S \rightarrow \epsilon$$

Derivation:

$$S \Rightarrow aSb$$
$$\Rightarrow aaSbb$$
$$\Rightarrow aabb$$

Derivation tree:



31 / 40

Ambiguity in natural languages

L'uomo guarda la donna con il binocolo

32 / 40

Ambiguity

- Grammar $\mathcal{G}$ is **ambiguous**
- iff
- there exists $w \in \mathcal{L}(\mathcal{G})$ that can be generated by two distinct canonical derivations, either both rightmost or both leftmost

33 / 40

Grammar for arithmetic expressions

- Grammar:
$$\begin{array}{lcl} E & \rightarrow & E + E \\ E & \rightarrow & E * E \\ E & \rightarrow & n \end{array}$$
- Is it ambiguous?
- Take $w = n + n * n$

34 / 40

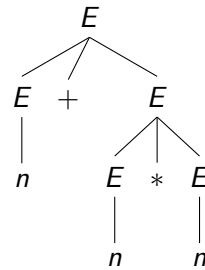
Grammar for arithmetic expressions

$$E \rightarrow E + E \mid E * E \mid n, w = n + n * n$$

Leftmost derivation:

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow n + E \\ &\Rightarrow n + E * E \\ &\Rightarrow n + n * E \\ &\Rightarrow n + n * n \end{aligned}$$

Derivation tree:



35 / 40

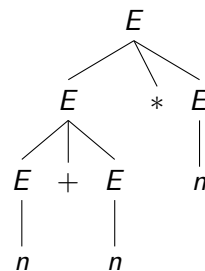
Grammar for arithmetic expressions

$$E \rightarrow E + E \mid E * E \mid n, w = n + n * n$$

This is yet another
leftmost derivation:

$$\begin{aligned} E &\Rightarrow E * E \\ &\Rightarrow E + E * E \\ &\Rightarrow n + E * E \\ &\Rightarrow n + n * E \\ &\Rightarrow n + n * n \end{aligned}$$

Derivation tree:

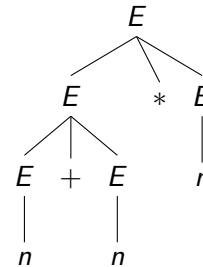
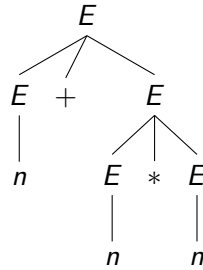


36 / 40

Grammar for arithmetic expressions

$$E \rightarrow E + E \mid E * E \mid n, w = n + n * n$$

For $w = n + n * n$ we get two distinct leftmost derivations, corresponding to the derivation trees below



The grammar is ambiguous!

Dangling else

- Grammar:

$$S \rightarrow \text{if } b \text{ then } S$$

$$S \rightarrow \text{if } b \text{ then } S \text{ else } S$$

$$S \rightarrow \text{other}$$

- Is it ambiguous?

- Take $w = \text{if } b \text{ then if } b \text{ then other else other}$

- Which of the two occurrences of “then” matches the single occurrence of “else”?

Dangling else

$w = \text{if } b \text{ then if } b \text{ then other else other}$

- One leftmost derivation:

$$\begin{aligned} S &\Rightarrow \text{if } b \text{ then } S \\ &\Rightarrow \text{if } b \text{ then if } b \text{ then } S \text{ else } S \\ &\Rightarrow \text{if } b \text{ then if } b \text{ then other else } S \\ &\Rightarrow \text{if } b \text{ then if } b \text{ then other else other} \end{aligned}$$

- Another leftmost derivation:

$$\begin{aligned} S &\Rightarrow \text{if } b \text{ then } S \text{ else } S \\ &\Rightarrow \text{if } b \text{ then if } b \text{ then } S \text{ else } S \\ &\Rightarrow \text{if } b \text{ then if } b \text{ then other else } S \\ &\Rightarrow \text{if } b \text{ then if } b \text{ then other else other} \end{aligned}$$

- Two different derivations (hence two different derivation trees)
- The grammar is ambiguous!

39 / 40

How bad this can get

- Ambiguity is undecidable
- No algorithm can be designed to decide whether a grammar is ambiguous or not
- But derivation trees are at the heart of syntax analysis!

40 / 40